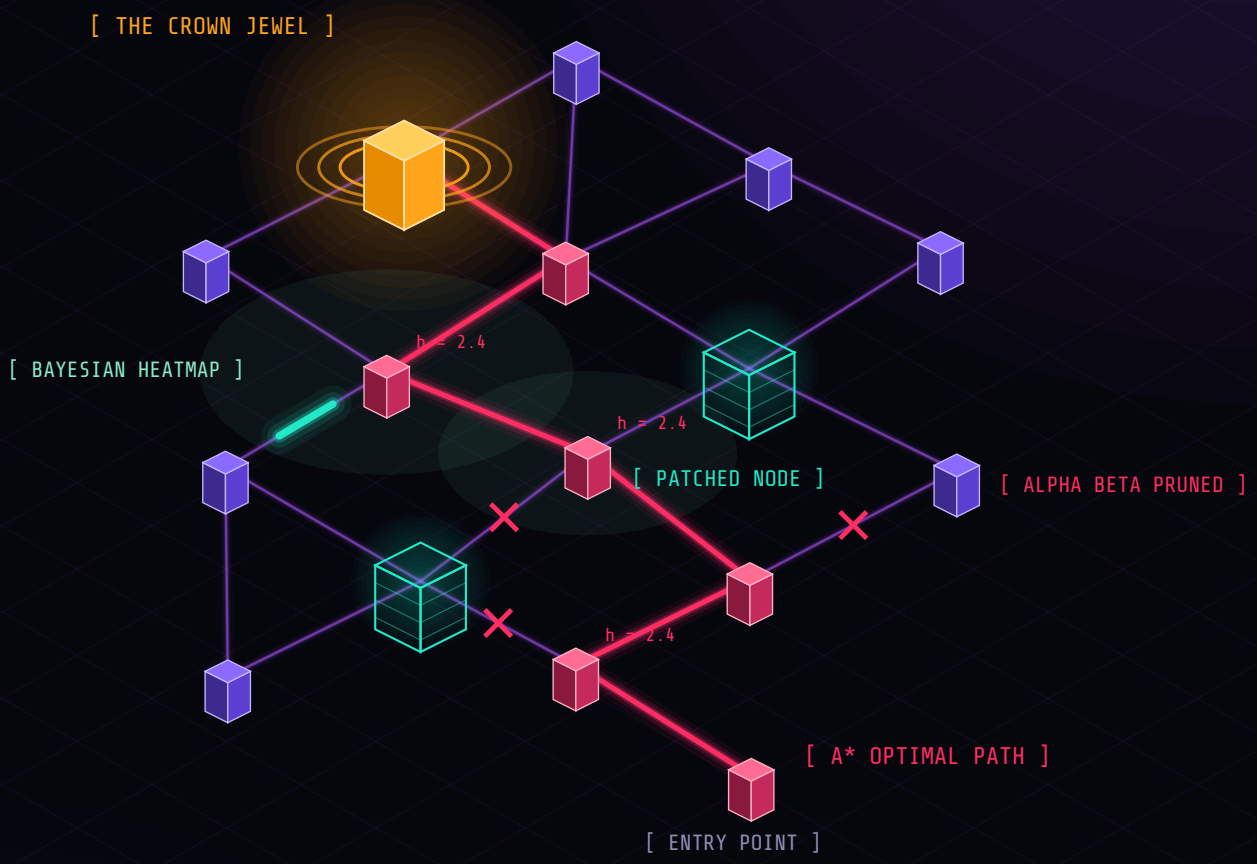
# CYBERSTRIKE

## NETWORK INFILTRATION GAME

AUTONOMOUS SYSTEM TRAFFIC

SYLLABUS TOPICS ARCHITECTURE




# BUILD PHASES PLAYBOOK

From an empty folder to a production grade adversarial AI system.

What we are building, how it works, the syllabus topics it covers, the engineering depth behind it, and seventeen build phases with concepts in easy words, completion checks and traps to avoid.

PYTHON 3 | PYGAME | NUMPY | MATPLOTLIB | SQLITE3 MULTIPROCESSING

v2.0

# 01 WHAT WE ARE BUILDING

The project in one page

CyberStrike is a turn based strategy game on the map of a company network. Machines are nodes and connections are links. An Attacker AI enters through one machine and tries to breach the Crown Jewel server. A Defender tries to stop it using patches, firewalls and honeypots, but it cannot see the Attacker. All it gets is noisy alerts from sensors, so it must reason about where the intruder probably is.

## CORE RULES

| ELEMENT | RULE |
|---|---|
| Map | Seeded network of 10 to 25 machines (default 15). Each machine has a hardness of 1 to 3 that sets how many turns a breach takes. |
| Attacker move | Advance to an adjacent machine. A harder machine holds it for more turns. |
| Defender actions | Patch (hardness plus 2), Firewall (blocks one link), Honeypot (hidden trap, reveals the intruder and delays it 2 turns), or Pass. Stock: 3 patches, 2 firewalls, 2 honeypots. |
| Sensors | One traffic record per machine each turn goes through a Naive Bayes classifier. Alerts can be false alarms or misses. |
| Winning | Attacker wins by finishing the breach of the Crown Jewel. Defender wins when turn 25 ends or no route remains open. |

## FOUR GAME MODES

- Human vs AI: you defend against the Minimax attacker (or attack against the CSP defender)
- AI vs AI: watch two intelligent agents fight, with a speed slider
- Human vs Human: hot seat on one screen
- Tutorial mode: pauses each turn and explains the AI decision in plain English

## WHY THIS PROJECT IS RARE

- Two different AIs fighting with hidden information
- Search, probability, learning, constraints and evolution in one connected system
- Every decision is visible on screen, so the AI explains itself
- Backed by real engineering: seeded tournaments, a database, charts, ablations

## NUMBERS THAT JUSTIFY THE AI

| NUMBER | WHAT IT MEANS |
|---|---|
| 11 million | ways to place 3 patches, 2 firewalls and 2 honeypots on a 15 machine map. Brute force is hopeless, so we use a CSP. |
| 74,000 | positions after only 3 rounds of attacker move plus defender reply. Alpha Beta pruning is what makes depth 6 affordable. |
| 0 | direct views of the intruder for the Defender. Everything comes from probability. |

# 02 HOW IT WORKS

One turn, from attacker thought to defender action

### 1 ATTACKER THINKS

A* finds the cheapest route; Minimax with Alpha Beta checks the defender replies and picks a move.

### 2 WORLD UPDATES

The rules engine applies the move and returns a brand new state. Breach timers and defenses update.

### 3 SENSORS FIRE

Every machine produces a traffic record. Noise, false alarms and misses are built in.

[ NEXT TURN REPEATS ]

LOOP UNTIL WIN

### 4 NAIVE BAYES

Classifies each record as normal or attack with Laplace smoothing and raises alert flags.

### 5 BELIEF UPDATE

Bayes rule turns alerts into a probability for each machine. Heatmap glows brighter where the intruder likely is.

### 6 CSP PLANS AND ACTS

Backtracking with MRV and MAC places a patch, firewall or honeypot under the rules, then the turn ends.

## AGENT SENSORS AND PEAS

| | ATTACKER | DEFENDER |
|---|---|---|
| Performance | Reach the Crown Jewel in the fewest turns, avoid honeypots | Stop the breach before turn 25, spend few defenses, keep a correct belief |
| Environment | Seeded network graph, hardness values, turn limit 25, noisy sensors | Same map, but the intruder is invisible to it |
| Actuators | Move to an adjacent machine | Place patch, firewall or honeypot, or pass |
| Sensors | Full topology, own position, visible defenses (honeypots hidden) | Topology, own defenses, one alert flag per machine; sees the intruder only on a honeypot hit |

## ENVIRONMENT TYPE, WITH REASONS

| PROPERTY | WHY |
|---|---|
| **Partially observable** | Defender cannot see the intruder; Attacker cannot see honeypots |
| **Stochastic percepts** | Rules are exact, but alerts are random draws with false alarms and misses |
| **Static** | Turn based: the world waits while an agent thinks (2 second budget) |
| **Discrete** | Machines, links, integer hardness, whole turns |
| **Multi agent, competitive** | Zero sum: an Attacker win is a Defender loss |

## READING THE LOOP

> The Attacker decides with look ahead, so it plans for the best defender reply, not just for its own path.

> The Defender never sees the attacker. It turns alerts into a belief, then plans placements that cover the likely routes. Honeypots are the only moment it gets certainty.

> Every step is written to an append only event log, so any game replays exactly and results flow into a SQLite database.

# 03 TOPICS COVERED

Every row maps a syllabus topic to a feature and a build phase

| SYLLABUS TOPIC | WHERE IT LIVES IN CYBERSTRIKE | PHASE |
|---|---|---|
| Intelligent agents, rationality, PEAS, environment types | Attacker and Defender agent design, environment classification | 01 |
| Uninformed search: BFS, DFS, uniform cost | Baseline route finders and benchmark table | 03 |
| Informed search, heuristics, A* | Attacker route planner, red optimal path with h values | 03 |
| Adversarial search: Minimax, Alpha Beta Pruning | Attacker decision engine with visible pruning | 04 |
| Basic probability, joint distributions, independence, Bayes rule | Classifier math and the belief update | 05 06 |
| Bayesian networks, Markov blankets | Belief tracker structure, heatmap explanations | 06 |
| Supervised learning, train validation test, K Fold, LOOCV, confusion matrix | Evaluation of the traffic classifier | 05 |
| Overfitting, underfitting, Ockham's razor | Feature set study, weight tuning on separate seeds | 05 09 |
| Naive Bayes and Laplace smoothing | Alert classifier on synthetic logs | 05 |
| CSP, backtracking, variable and value ordering, forward checking, MAC | Defender resource planner and solver benchmark | 07 |
| Genetic algorithm | Evolving balanced network maps | 08 |
| Linear regression | Difficulty predictor used as the fast fitness function | 08 |
| Local search and optimisation problems, hill climbing | Evaluation weight tuning and the post game coach | 09 |

## SAME MAP, BY SYLLABUS WEEK

| WEEK | SYLLABUS BLOCK | PHASE |
|---|---|---|
| 1 to 2 | What is AI, intelligent agents, environments, structure of agents | 01 |
| 2 to 3 | Problem solving by searching, uninformed, informed, heuristics | 03 |
| 4 to 5 | Local search, optimisation, genetic algorithm | 08 09 |
| 7 to 8 | CSPs, backtracking, forward checking, MAC, Minimax, Alpha Beta | 04 07 |
| 9 | Probability, Bayes rule, Bayesian networks, Markov blankets | 05 06 |
| 10 to 11 | Supervised learning, K Fold, LOOCV, Naive Bayes, Laplace, linear regression | 05 08 |

## COVERAGE PROMISE

Thirteen syllabus rows, all inside one running game. If time gets tight, drop Phase 9 first and Phase 8 second. The CORE phases alone still cover agents, search, adversarial search, probability, Bayesian networks, supervised learning, Naive Bayes and CSPs.

# 04 ENGINEERING DEPTH

Real backend, database, operating system and computer science ideas inside the game

## BACKEND AND SOFTWARE ENGINEERING

| CONCEPT | HOW CYBERSTRIKE USES IT | PHASE |
|---|---|---|
| **Layered architecture** | Core never imports UI or AI code, and the UI never edits rules | 00 01 |
| **Design patterns** | Strategy for agents, Command for actions, Observer for the UI, Factory for agent creation | 01 15 |
| **Event sourcing** | The game is a stream of events; replay rebuilds any state exactly | 01 10 |
| **Deterministic simulation** | Seeded random streams per game and per worker | 00 11 |
| **Typed, tested code** | Type hints, dataclasses, unit tests that run on every push | 00 |
| **Logging, CLI, profiling** | JSON log lines, argparse commands, cProfile tables | 00 03 12 |

## DATABASES

| CONCEPT | HOW CYBERSTRIKE USES IT | PHASE |
|---|---|---|
| **Relational schema design** | Normalised tables with keys and constraints | 10 |
| **Transactions and WAL mode** | Batched inserts, readers not blocked by the writer | 10 11 |
| **Indexes and query plans** | EXPLAIN QUERY PLAN compared on 100,000 rows | 10 |
| **SQL analytics** | GROUP BY win rates and window function rolling averages | 10 16 |
| **Migrations** | Numbered SQL upgrades with a schema version table | 10 |

## OPERATING SYSTEMS

| CONCEPT | HOW CYBERSTRIKE USES IT | PHASE |
|---|---|---|
| **Processes, threads and the GIL** | AI search runs in a worker process so the window never freezes | 11 |
| **Message passing and shared memory** | Queues and pipes for messages, shared_memory for big arrays | 11 |
| **Scheduling and worker pools** | Tournament jobs spread over all cores in chunks | 11 |
| **Timeouts and termination** | A hard time limit per move enforced by the parent process | 11 |
| **Atomic files and memory mapping** | Write a temp file then rename; memory mapped timing logs | 11 |
| **Signals and clean shutdown** | Join workers and unlink shared memory | 11 |

# 04 ENGINEERING DEPTH

Algorithms, data structures, AI and ML engineering, security engineering

## COMPUTER SCIENCE ALGORITHMS AND DATA STRUCTURES

| CONCEPT | HOW CYBERSTRIKE USES IT | PHASE |
|---|---|---|
| **Priority queue (binary heap)** | The open list of A* built on heapq | 03 |
| **Hashing and caching** | Zobrist hash and a transposition table with a replacement policy | 12 |
| **Bit manipulation** | Machine sets stored as bitmasks | 12 |
| **Memoisation** | Distance tables and cached search results | 03 12 |
| **Graph theory** | Connectivity, articulation points and minimum cut for chokepoints | 01 14 |
| **Constraint propagation** | AC 3 and MAC inside the CSP planner | 07 |
| **Search complexity** | Nodes expanded measured against the b to the d growth and pruning | 04 |

## AI AND ML ENGINEERING

| CONCEPT | HOW CYBERSTRIKE USES IT | PHASE |
|---|---|---|
| **Calibration and Brier score** | Honest probabilities for the belief tracker | 13 |
| **ROC, precision recall, cost based thresholds** | Alert threshold from the cost of a miss versus a false alarm | 13 14 |
| **Feature selection and explanation** | Mutual information and per feature contributions | 13 |
| **Exact versus approximate inference** | Bayes filter compared with a particle filter | 06 13 |
| **Monte Carlo methods** | MCTS challenger attacker | 13 |
| **Evolutionary computation** | Genetic algorithm with a regression surrogate | 08 |
| **Statistics for agents** | Elo ladder, bootstrap intervals, paired tests | 16 |
| **Bias and variance** | Learning curves against training size | 13 |

## SECURITY ENGINEERING (SIMULATION ONLY)

| CONCEPT | HOW CYBERSTRIKE USES IT | PHASE |
|---|---|---|
| **Threat modelling** | STRIDE document for the fictional network | 14 |
| **CVSS style scoring** | Hardness derived from complexity, privileges and exposure | 14 |
| **Kill chain and ATT&CK stages** | Attacker moves labelled by stage | 14 15 |
| **Defense in depth and deception** | Patch, segment and honeypot layers | 07 14 |
| **Detection cost trade offs** | Missed attack versus false alarm cost model | 13 14 |

All security content is a teaching simulation: no real exploits, scans or network traffic are ever used.

# 05 ARCHITECTURE

Layers, dependencies and folder layout

**UI LAYER**
Pygame window, menus, neon theme, heatmap, panels, replay viewer
`screens` `panels` `heatmap` `replay`

**AGENTS**
One interface choose_action(observation)
`Random` `Greedy A*` `Minimax` `MCTS` `Belief CSP` `Human`

**AI BRAINS**
Independent, testable packages
`search` `prob` `csp` `ml` `evo`

**PLATFORM**
Event store, SQLite, worker processes, shared memory, logging
`events` `sqlite` `workers` `shm` `logging`

**GAME CORE**
Pure Python rules. Never imports Pygame, so it runs headless
`network` `state` `rules` `generator`

**DATA AND PROOF**
Seeds, maps, results, tournament, charts, Elo
`data` `results` `tournament` `charts`

## FOLDER LAYOUT

```
cyberstrike/  main.py config.py README.md
  core/     network state rules generator
            events eventstore bitset roles
            scoring stages chokepoints replay
  agents/   base random_agents greedy_attacker
            minimax_attacker mcts_attacker
            belief_csp_defender human coach
  search/   astar uninformed minimax alphabeta
            evaluation zobrist transposition
  prob/     belief bayesnet blanket particle
  csp/      model solver arc
  ml/       synthetic_logs naive_bayes metrics
            crossval regression sensor
            calibration curves features
  evo/      genetic repair localsearch
  db/       schema.sql db.py migrations
  runtime/  ai_worker ipc shm tournament_pool
  ui/       app theme draw panels heatmap
            menu screens tutorial ga_panel tray
  experiments/ tournament charts ablation elo
            bench_search bench_csp tune_weights
  data/  results/  tests/  docs/
```

## DEPENDENCY RULES

- Rule 1: core imports nothing from the other folders
- Rule 2: ui may import core and agents, never the reverse
- Rule 3: platform code (db, runtime) wraps the core, never leaks into it
- Rule 4: every package has its own tests in tests/
- Rule 5: one seeded generator for all randomness
- Rule 6: every experiment runs headless and writes to the database

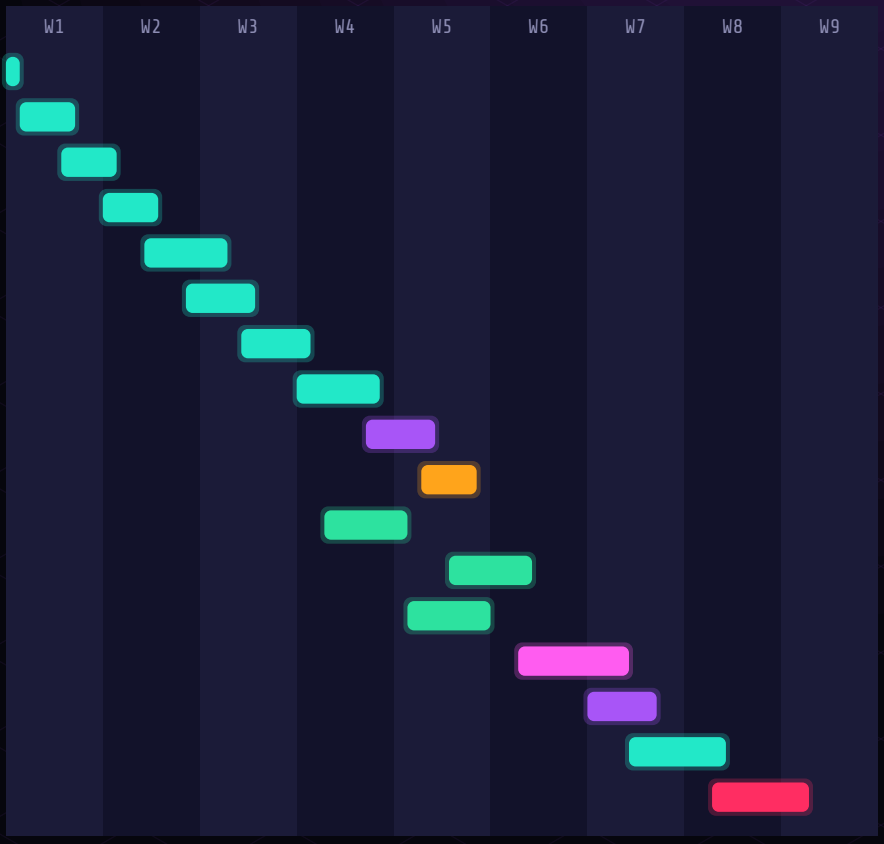
## WHY THIS STRUCTURE WORKS

Rules never touch the screen, so one engine runs in a glowing window and in a background tournament of thousands of games. Every AI sits behind one agent interface, so any mix of players works and every comparison swaps one object. The platform layer (event store, database, worker processes) wraps the core and never leaks into it.

# 06 BUILD ROADMAP

Seventeen phases, three routes, scale it to your deadline

00 Setup
01 Game world and rules
02 Pygame battlefield
03 A* and search
04 Minimax and Alpha Beta
05 Naive Bayes sensors
06 Bayesian belief
07 CSP planner
08 Genetic algorithm
09 Local search
10 Event log and database
11 Processes and concurrency
12 Performance engine
13 AI lab
14 Security realism
15 Integration
16 Experiments




CORE: must ship
SYSTEMS: real engineering
BOOST: adds topics
LAB: beyond the syllabus
STRETCH: only if ahead
FINAL: proof and packaging

## CHOOSE YOUR ROUTE

| ROUTE | LENGTH | PHASES | WHAT IT DEMONSTRATES |
|---|---|---|---|
| A FOCUSED | 5 weeks | Phases 0 to 7, then 10, 15 and 16 | A complete adversarial AI game with search, probability, learning and constraints, plus a replayable event log and a database |
| B BALANCED | 7 weeks | Route A plus phases 8, 11 and 12 | Adds evolution, multi core processing and a measured performance engine |
| C FULL | 9 weeks | Everything, phases 0 to 16 | Adds local search, calibration, particle filter, MCTS, an Elo ladder and a security grounded model |

Pick the route that matches your real deadline now, and keep the order of phases. A shorter route simply stops earlier; nothing already built is thrown away.

# 06 DELIVERABLES BY PHASE

What you can show at the end of every phase

| # | PHASE | DELIVERABLE YOU CAN SHOW | WHEN |
|---|---|---|---|
| 00 | **Setup** | Repository, environment, folder layout and a blank Pygame window | DAY 1 |
| 01 | **Game world and rules** | Complete game logic playable in the terminal, 25 plus unit tests | DAY 2 TO 5 |
| 02 | **Pygame battlefield** | Neon themed playable window: human defender against a random attacker | DAY 5 TO 8 |
| 03 | **A* and search** | Greedy A* attacker with red path overlay and a search benchmark table | DAY 8 TO 11 |
| 04 | **Minimax and Alpha Beta** | Minimax and Alpha Beta attacker with visible pruning and nodes table | DAY 11 TO 16 |
| 05 | **Naive Bayes sensors** | Traffic classifier with K Fold, confusion matrix, charts and alert sensor | DAY 14 TO 18 |
| 06 | **Bayesian belief** | Live Bayesian heatmap showing where the intruder probably is | DAY 18 TO 22 |
| 07 | **CSP planner** | Belief driven CSP defender plus a solver comparison chart | DAY 22 TO 27 |
| 08 | **Genetic algorithm** | Evolved fair maps, regression difficulty predictor and fitness panel | DAY 27 TO 31 |
| 09 | **Local search** | Tuned attacker weights and a post game coach with live diagnostics tray | DAY 31 TO 34 |
| 10 | **Event log and database** | Replayable event log, SQLite schema, indexes, migrations and SQL analytics | DAY 24 TO 29 |
| 11 | **Processes and concurrency** | AI in its own process, multi core tournament runner, speedup chart | DAY 33 TO 38 |
| 12 | **Performance engine** | Transposition table, bitmask states and profiler tables with measured speedups | DAY 30 TO 35 |
| 13 | **AI lab** | Calibrated sensors, particle filter, MCTS challenger and an Elo ladder | DAY 38 TO 45 |
| 14 | **Security realism** | Threat model, CVSS style hardness, stage labels, chokepoint analysis and cost model | DAY 43 TO 47 |
| 15 | **Integration** | Menu with four modes, tutorial captions and all reference panels | DAY 46 TO 52 |
| 16 | **Experiments** | Ten charts, Elo ladder, ablations, README, demo video and a one page project brief | DAY 52 TO 58 |

PHASE

00

# SETUP AND SKELETON

BUILD PROGRESS

WHEN DAY 1 | OWNER BOTH | TIER CORE

MISSION Create a clean home for the project so every later phase plugs in without rewriting. One hour of discipline now saves days later.

## YOU WILL BUILD

- Create the Git repository, a .gitignore and a README stub with the project name and the four tools
- Create a virtual environment, activate it, install pygame, numpy and matplotlib with pip, and freeze the versions into requirements.txt
- Create the folder layout from the Architecture page with an empty __init__.py in every package
- Write config.py holding every constant: turn limit 25, defender stock 3 patches, 2 firewalls, 2 honeypots, map size 15, master seed 42, theme colors
- Write utils/rng.py so the whole project draws random numbers from one seeded generator
- Open a blank Pygame window titled CyberStrike and close it cleanly on Escape
- Add structured JSON logging (one line per event) and an argparse command line with play, tournament and replay commands

## CONCEPTS IN EASY WORDS

**Seed.** A seed fixes the random number stream, so the same seed always gives the same map and the same alerts. That is what makes experiments repeatable and comparisons fair.

**Headless first.** Game rules never import Pygame. That lets AI play thousands of games in the background without a window, which experiments need.

**No magic numbers.** A value like 25 buried in code is a bug waiting to happen. Name it once in config.py and import it everywhere.

**Structured logging.** Logs written as JSON lines can be searched and loaded into a database later. Free text cannot.

## DEFINITION OF DONE

- [ ] Window opens and closes without errors
- [ ] Same seed prints the same five random numbers on two separate runs
- [ ] Smoke test passes using unittest from the standard library
- [ ] First commit pushed and tagged phase0

## TRAPS TO AVOID

- Installing packages globally instead of inside the virtual environment
- Scattering constants across files instead of keeping them in config.py

## MILESTONE TO DEMONSTRATE

Repository, environment, folder layout and a blank Pygame window.

PHASE

# 01 GAME WORLD AND RULES

BUILD PROGRESS

WHEN DAY 2 TO 5 | OWNER LEADER | TIER CORE

MISSION Build the entire game as pure Python logic and play it in the terminal before drawing a single pixel.

## YOU WILL BUILD

- network.py: graph as an adjacency dictionary, hardness per machine (1 to 3), entry machine and crown jewel
- state.py: GameState with attacker position, breach turns left, patched set, firewalled links, hidden honeypots, stock, turn number and side to move
- rules.py: legal_actions(state), apply(state, action) which returns a NEW state, is_terminal(state) and utility(state)
- generator.py: seeded random connected map, with a check that two separate routes exist from entry to crown jewel
- Random attacker and random defender, plus a text printer that shows each turn
- A function that builds the defender observation (alerts only) separately from the full state
- Represent every move as a small Command object (a dataclass) so it can be logged, replayed and undone

## CONCEPTS IN EASY WORDS

**Agent and PEAS.** An agent senses and acts. PEAS means Performance, Environment, Actuators, Sensors. Write one for the attacker and one for the defender before coding; it becomes your agent interface.

**State versus observation.** The state is the whole truth. The observation is what one agent may see. The defender observation hides the attacker, which is why probability is needed later.

**Immutable moves.** apply returns a new state instead of editing the old one. Search can try a move, look at the result and throw it away with no undo bugs.

**Command pattern.** Wrap each action as an object. Objects can be saved, replayed, sent between processes and unit tested. This is the root of the event log in Phase 10.

## DEFINITION OF DONE

- [ ] 1,000 random games finish with no crash
- [ ] At least 25 unit tests cover moves, patches, firewalls, honeypots and win checks
- [ ] Generator never returns a disconnected map over 500 seeds
- [ ] Environment classified on all five dimensions in docs/peas.md

## TRAPS TO AVOID

- Changing the state inside search code; always copy instead
- Letting the defender observation leak the attacker position

## MILESTONE TO DEMONSTRATE

Complete game logic playable in the terminal, 25 plus unit tests.

PHASE

02

BUILD PROGRESS

# PYGAME BATTLEFIELD

WHEN DAY 5 TO 8 | OWNER MEMBER 2 | TIER CORE

MISSION Give the game a face. Build the dark neon battlefield from the reference image and let a human play the Defender with the mouse.

## YOU WILL BUILD

- ui/theme.py with the exact palette of this document, fonts and glow helpers
- Main loop at 60 frames per second: handle events, update, draw
- Draw machines as glowing server cubes, links as neon lines, the crown jewel with orange rings, patched nodes in cyan wireframe
- Defender Command panel on the right: stock bars for patches, firewalls and honeypots, plus action buttons
- Mouse picking: click a machine or link to place the selected defense and highlight legal targets
- Smooth attacker movement with easing so moves are readable, plus a message log

## CONCEPTS IN EASY WORDS

**Game loop.** Every frame does three things: read input, update the world, draw. Rules change only on turns; drawing runs every frame.

**Model and view.** The screen reads the state and draws it. It never edits rules. Swap the screen later and the game still works.

**Glow effect.** Draw the same shape several times, larger and fainter, on a surface with alpha. It is cheap and gives the neon look.

## DEFINITION OF DONE

- [ ] A human finishes a full game against the random attacker using only the mouse
- [ ] Stays near 60 fps with 20 machines
- [ ] Honeypots stay hidden until triggered
- [ ] Screenshot saved for the README

## TRAPS TO AVOID

- Running AI computation inside the draw function, which freezes the window
- Hard coding pixel positions instead of computing them from the graph

## MILESTONE TO DEMONSTRATE

Neon themed playable window: human defender against a random attacker.

PHASE

03

BUILD PROGRESS

# ATTACKER BRAIN I: A*

WHEN DAY 8 TO 11 | OWNER LEADER | TIER CORE

MISSION Teach the attacker to find the cheapest route to the crown jewel, and prove the search is optimal.

## YOU WILL BUILD

- BFS and DFS baselines to see uninformed search, and uniform cost search (Dijkstra) where entering a machine costs its hardness
- A* with f = g + h, where h comes from a distance table computed once on the clean map
- A counter object that records nodes expanded and time
- GreedyAttacker: runs A* every turn from its current position and follows the first step
- Red path overlay with h values on each machine, toggled by a key
- Benchmark script: BFS, uniform cost and A* on 100 maps, printed as a table
- Use heapq as the priority queue and profile it with cProfile

## CONCEPTS IN EASY WORDS

**Uninformed versus informed.** Uninformed search knows only the cost so far. Informed search adds a guess h of the cost still to go, so it heads toward the goal.

**A star.** Always expand the machine with the lowest f = g + h. g is the cost already paid, h is the estimated cost left.

**Admissible and consistent.** Admissible: h never overestimates, so the first goal found is cheapest. Consistent: h never drops by more than the step cost, so nothing is reopened. Defenses only raise costs, so distances from the clean map stay safe.

**Binary heap.** A heap gives push and pop in O(log n). A plain sorted list would make the open list of A* the slowest part of the program.

## DEFINITION OF DONE

- [ ] A* cost equals uniform cost search cost on all 100 maps
- [ ] A* expands fewer machines than BFS and uniform cost, shown in a table
- [ ] Path overlay works in the Pygame window
- [ ] Admissibility argument written in docs

## TRAPS TO AVOID

- Using a heuristic that overestimates cost, which breaks optimality
- Forgetting to count nodes expanded, leaving nothing to compare later

## MILESTONE TO DEMONSTRATE

Greedy A* attacker with red path overlay and a search benchmark table.

PHASE

04

BUILD PROGRESS

# ATTACKER BRAIN II: MINIMAX

WHEN DAY 11 TO 16 | OWNER LEADER | TIER CORE

MISSION Make the attacker think about the defender's best replies, and make the pruning visible on screen.

## YOU WILL BUILD

- minimax(state, depth, side) alternating attacker and defender plies, returning a value and the best action
- evaluation(state, weights): negated cheapest breach cost, routes still open, honeypot risk, defender stock left
- Alpha Beta with the same interface plus move ordering: try actions along the A* path first
- Depth limit of 6 plies, iterative deepening with a 2 second budget, fall back to the last finished depth
- Defender replies limited to machines within two links of the attacker, keeping branching near 14
- Overlay: pruned branches in dashed red labelled ALPHA BETA PRUNED, plus a live nodes expanded counter
- Game tree printer for tiny positions, used in the documentation

## CONCEPTS IN EASY WORDS

**Game tree.** Nodes are positions. You move, the opponent replies, you move again. Minimax assumes the opponent always plays the move that is worst for you.

**Alpha and beta.** Alpha is the best value the maximiser can already guarantee, beta the best the minimiser already holds. When alpha reaches beta the rest of that branch cannot change the decision, so skip it.

**Move ordering.** With perfect ordering Alpha Beta searches about the square root of the nodes plain Minimax needs. Good ordering is the whole game.

**Evaluation function.** At the depth limit we cannot see the end, so a score estimates how good the position is. Better features make a smarter attacker.

## DEFINITION OF DONE

- [ ] Minimax and Alpha Beta choose identical moves on 100 positions
- [ ] Nodes expanded table for depths 2 to 6
- [ ] Every attacker move finishes inside 2 seconds
- [ ] Minimax attacker beats the greedy A* attacker against a static defender

## TRAPS TO AVOID

- Searching the full defender action list; restrict it to nearby machines
- Comparing Alpha Beta with plain Minimax at different depths

## MILESTONE TO DEMONSTRATE

Minimax and Alpha Beta attacker with visible pruning and nodes table.

PHASE

05

BUILD PROGRESS

# DEFENDER SENSES I: NAIVE BAYES

WHEN DAY 14 TO 18 | OWNER MEMBER 2 | TIER CORE

MISSION Build the Defender's sensors: a classifier that reads traffic records and raises alerts, and measure it honestly.

## YOU WILL BUILD

- synthetic_logs.py generates synthetic_logs.csv: protocol, port group, bytes group, failed logins, hour group, label normal or attack, with deliberate overlap and noise
- naive_bayes.py: fit from counts, predict with a sum of logarithms, Laplace smoothing with alpha 1
- Split into train, validation and test; tune only on validation
- K Fold with K 5, plus LOOCV on a small subset
- Confusion matrix, accuracy, precision, recall and F1 drawn with Matplotlib in the theme colors
- Overfitting study: small feature set versus full set versus no smoothing
- SensorSimulator: each turn give every machine a traffic record (attack style where the intruder is) and return alert flags

## CONCEPTS IN EASY WORDS

**Naive Bayes.** Bayes rule plus one naive assumption: features are independent once the class is known. Score each class by prior times the product of feature probabilities and pick the larger.

**Zero count problem.** An unseen value gives probability 0 and wipes out the product. Laplace smoothing adds 1 to every count so nothing is ever exactly 0.

**K Fold and confusion matrix.** Split into K parts, train on K minus 1, test on the last, rotate K times and average. The confusion matrix counts hits and false alarms: recall is attacks caught, false alarm rate is normal traffic flagged.

**Ockham's razor.** If two models do equally well on validation data, keep the simpler one. Extra complexity mostly buys overfitting.

## DEFINITION OF DONE

- [ ] 5 fold results table with and without Laplace smoothing
- [ ] Confusion matrix and F1 chart saved in results/
- [ ] Recall and false alarm rate exported to config for Phase 6
- [ ] Optional upgrade: same code runs on the real NSL KDD file

## TRAPS TO AVOID

- Tuning on the test split
- Multiplying raw probabilities instead of adding logarithms

## MILESTONE TO DEMONSTRATE

Traffic classifier with K Fold, confusion matrix, charts and alert sensor.

PHASE

06

BUILD PROGRESS

# DEFENDER SENSES II: BELIEF

WHEN DAY 18 TO 22 | OWNER MEMBER 2 | TIER CORE

MISSION Turn noisy alerts into a probability for each machine, shown as a live heatmap.

## YOU WILL BUILD

- BeliefTracker holding N probabilities, starting at the entry machine
- Predict step: movement model, stay with some probability or move to a neighbour, weighted by hardness
- Update step: multiply each machine by the alert likelihood (recall if an alert is present there, false alarm rate otherwise), then normalise to sum 1
- markov_blanket(machine) returning parents, children and the children's other parents
- Pygame heatmap: soft glowing discs sized by probability, plus a top 3 list
- Honeypot rule: when a honeypot triggers, set belief to 1 on that machine
- Exact enumeration check on a 6 machine map to prove the tracker is correct

## CONCEPTS IN EASY WORDS

**Belief.** Your probability for each place the intruder could be, given every alert so far.

**Bayes update, worked.** Two machines, prior 0.5 each. An alert fires on A. Alerts fire 80 percent of the time when the intruder is there and 10 percent when not. A becomes 0.5 times 0.8 divided by (0.5 times 0.8 plus 0.5 times 0.1), about 0.89.

**Bayesian network and Markov blanket.** A full joint table grows exponentially. A Bayesian network stores only small local tables. Given its Markov blanket, a variable is independent of everything else, so only neighbours matter.

**Predict then correct.** Each turn: guess where the intruder moved, then fix the guess with the new evidence.

## DEFINITION OF DONE

- [ ] Hand worked example matches the code to 4 decimals
- [ ] Probability on the true machine clearly beats the uniform baseline (chart)
- [ ] Heatmap updates every turn without frame drops
- [ ] Tracker agrees with exact enumeration on 6 machine maps

## TRAPS TO AVOID

- Forgetting to normalise, so beliefs stop summing to 1
- Letting a belief hit exactly 0, which can never recover; keep a tiny floor

## MILESTONE TO DEMONSTRATE

Live Bayesian heatmap showing where the intruder probably is.

PHASE

07

BUILD PROGRESS

# DEFENDER BRAIN: CSP PLANNER

WHEN DAY 22 TO 27 | OWNER MEMBER 2 | TIER CORE

MISSION Let the Defender plan all its defenses as a constraint problem, and prove that smarter solving is faster.

## YOU WILL BUILD

- model.py: variables P1, P2, P3, F1, F2, H1, H2; domains are machines for patches and honeypots, links for firewalls
- Constraints: all different within a type; no honeypot on entry or crown jewel; no honeypot on a patched machine; each of the 3 cheapest routes holds at least one patch or firewall
- solver.py with switches: plain backtracking, MRV, least constraining value using belief, forward checking, MAC with AC 3
- Counters for assignments tried, backtracks and time
- BeliefCSPDefender: re solve every turn with the latest belief, then execute the most urgent unit
- HUD: stock bars and a CSP solver status light
- Benchmark on 100 instances at map sizes 10, 15 and 20

## CONCEPTS IN EASY WORDS

**CSP.** Variables, domains, constraints. Find values for every variable so that no constraint breaks. Same family as map colouring, N Queens and Sudoku.

**Backtracking.** Assign one variable, go deeper, undo when stuck.

**MRV and LCV.** MRV picks the variable with the fewest legal values, so failures appear early. LCV tries the value that removes the fewest options from others.

**Forward checking and MAC.** Forward checking deletes conflicting values from neighbours after each assignment and backtracks if any domain empties. MAC keeps every arc consistent, catching dead ends even earlier.

## DEFINITION OF DONE

- [ ] All four solver settings return legal plans
- [ ] Chart: MAC needs fewer backtracks than plain backtracking at every map size
- [ ] Belief plus CSP defender beats the static defender over 300 maps
- [ ] Backtracking dry run on a 4 region map colouring written in docs

## TRAPS TO AVOID

- Writing constraints that make every problem unsolvable; test on tiny cases first
- Timing solvers without fixed seeds, so charts are not repeatable

## MILESTONE TO DEMONSTRATE

Belief driven CSP defender plus a solver comparison chart.

PHASE

08

BUILD PROGRESS

# GENETIC ALGORITHM: MAPS

WHEN DAY 27 TO 31 | OWNER LEADER | TIER BOOST

MISSION Evolve fair, interesting maps automatically, using a fast regression model as the judge.

## YOU WILL BUILD

- Chromosome: hardness list, edge on or off list, entry index and crown jewel index
- Repair function that reconnects the graph so every child is valid
- Tournament selection, single point crossover on the edge list, mutation that flips an edge or changes a hardness, elitism to keep the best
- regression.py: simulate 500 random maps, extract features (machines, links, average hardness, cheapest cost, route count) and fit linear regression by least squares in NumPy to predict attacker turns to breach
- Fitness: predicted attacker win rate near 50 percent plus a bonus for route variety
- UI panel with fitness bars and generation counter as in the reference image; export best maps to data/maps
- Validate the regression on held out maps and report R squared

## CONCEPTS IN EASY WORDS

**Genetic algorithm.** Keep a population of candidate solutions. Each generation: pick the fittest, mix pairs (crossover), randomly tweak some (mutation), repeat.

**Local versus global.** Hill climbing improves one candidate and can get stuck. A GA keeps many candidates, so it explores wider.

**Linear regression.** Fit the line or plane that minimises squared error between predictions and data. It judges a map in microseconds instead of playing a whole game.

## DEFINITION OF DONE

- [ ] Average fitness rises over 40 generations (chart)
- [ ] Evolved maps give win rates nearer 50 percent than random maps
- [ ] R squared reported on held out maps
- [ ] Evolved maps selectable in the game menu

## TRAPS TO AVOID

- Crossover that creates disconnected maps because the repair step is missing
- Playing real games to score every candidate, which is far too slow

## MILESTONE TO DEMONSTRATE

Evolved fair maps, regression difficulty predictor and fitness panel.

PHASE

09

BUILD PROGRESS

# LOCAL SEARCH: TUNING AND COACH

WHEN DAY 31 TO 34 | OWNER LEADER | TIER STRETCH

MISSION Tune the attacker brain with hill climbing and build a post game coach that suggests better defender moves.

## YOU WILL BUILD

- Weight vector for the 4 evaluation features; a neighbour is one weight nudged up or down
- Hill climbing with random restarts, and optional simulated annealing
- Score each candidate by win rate over training seeds 0 to 99; report only on test seeds from 1000 up
- Coach: after a game replay each defender decision, search nearby alternative actions, report the best one and the turns it would have gained
- Bottom tray in the UI printing live local search diagnostics as in the reference image
- Plot score per iteration

## CONCEPTS IN EASY WORDS

**Local search.** Start somewhere, look at the neighbours, step to a better one, stop when none is better.

**Local maximum.** A peak that is not the highest. Fix it with random restarts or by sometimes accepting a worse move (simulated annealing).

**Train versus test.** Tune on training seeds only. Touch test seeds once at the end, otherwise you are fooling yourself.

## DEFINITION OF DONE

- [ ] Tuned weights beat default weights on test seeds
- [ ] Coach produces advice for a finished game
- [ ] Iteration curve saved in results/
- [ ] Training and test seeds kept separate in code

## TRAPS TO AVOID

- Reporting results on the same seeds used for tuning
- Step sizes so large the climb jumps over peaks

## MILESTONE TO DEMONSTRATE

Tuned attacker weights and a post game coach with live diagnostics tray.

PHASE

10

BUILD PROGRESS

# EVENT SOURCING AND DATABASE

WHEN DAY 24 TO 29 | OWNER MEMBER 2 | TIER SYSTEMS

MISSION Store every game as an unchangeable stream of events, then keep results in a real relational database, so any game replays exactly and any question is answered with SQL.

## YOU WILL BUILD

- events.py: dataclass events (GameStarted, AttackerMoved, DefensePlaced, AlertsRaised, BeliefUpdated, GameEnded) each with a version number and timestamp
- EventStore: append only, one JSON line per event in a file; rebuild any state by folding the events from the start (replay)
- schema.sql with tables maps, games, moves, alerts, beliefs, matchups and experiments; primary keys, foreign keys, NOT NULL and CHECK constraints
- db.py using sqlite3 from the standard library: one connection per process, WAL mode, foreign keys on, batch inserts inside one transaction
- Indexes on game id, seed and matchup; use EXPLAIN QUERY PLAN to show the speed gain on 100,000 rows
- migrations/: numbered SQL files plus a schema_version table so the database upgrades safely
- Analytics queries: win rate per matchup with GROUP BY, rolling average with a window function, ten slowest moves

## CONCEPTS IN EASY WORDS

**Event sourcing.** Do not store the current state, store everything that happened. State is the result of replaying events. You get replay, undo, debugging and an audit trail for free.

**Keys and normalisation.** Put each fact in one place and link tables with keys so the data cannot contradict itself. A foreign key stops you saving a move for a game that does not exist.

**Transactions and ACID.** A transaction groups writes so all of them happen or none do. One transaction with 1,000 inserts is often far faster than 1,000 separate commits.

**Indexes.** An index is a sorted lookup structure (a B tree). Queries on indexed columns skip scanning the whole table. It costs space and slows writes, so index only what you query.

## DEFINITION OF DONE

- [ ] Any finished game replays from its event file and ends in the identical final state
- [ ] Database holds 10,000 games with foreign keys enforced and no errors
- [ ] EXPLAIN QUERY PLAN output saved, showing a table scan versus an index search
- [ ] Win rate per matchup from one SQL query matches the Python result

## TRAPS TO AVOID

- Storing everything as one JSON blob instead of columns for what you query
- Committing after every single insert

## MILESTONE TO DEMONSTRATE

Replayable event log, SQLite schema, indexes, migrations and SQL analytics.

PHASE

11

# PROCESSES AND CONCURRENCY

BUILD PROGRESS

WHEN DAY 33 TO 38 | OWNER LEADER | TIER SYSTEMS

MISSION Keep the window responsive and the experiments fast with real operating system ideas: processes, message passing, shared memory and hard timeouts.

## YOU WILL BUILD

- ai_worker.py: a separate process that receives an observation over a queue, thinks, and sends back an action
- Time budget: if the worker passes 2 seconds the parent terminates it and uses the last finished depth, which the worker sent through a pipe
- The Pygame loop polls the result queue every frame without blocking and shows a thinking animation
- tournament_pool.py: multiprocessing Pool where one job is one seeded game; jobs are chunked, and one writer process receives results through a queue and writes to the database
- Share the distance table and map arrays between processes with multiprocessing.shared_memory as NumPy views, without copying
- Independent random streams per worker with NumPy SeedSequence spawn, so parallel results equal serial results
- Clean shutdown: handle signals, join workers, unlink shared memory; atomic result files (write a temp file then rename) and a memory mapped array for per move timing logs

## CONCEPTS IN EASY WORDS

**Process versus thread.** A process has its own memory; threads share memory. Python's GIL lets one thread run Python code at a time, so CPU heavy search needs separate processes to use many cores.

**Inter process communication.** Processes cannot read each other's memory, so they exchange messages through queues and pipes, or share one memory block.

**Timeouts.** You cannot safely interrupt a running function, but you can terminate a whole process. That gives a real hard time limit.

**Single writer.** Many writers fighting over one file or database cause locks. One writer fed by a queue removes the problem.

## DEFINITION OF DONE

- [ ] Window stays near 60 fps while the AI thinks for 2 seconds
- [ ] A killed or slow worker never freezes the game
- [ ] 300 games run at least 3 times faster on 4 cores than on 1 (speedup chart)
- [ ] Parallel run gives identical results to the serial run for the same seeds

## TRAPS TO AVOID

- Sharing one random generator between workers, which breaks reproducibility
- Forgetting that Windows starts workers by re importing your modules: guard the entry code with a main check

## MILESTONE TO DEMONSTRATE

AI in its own process, multi core tournament runner, speedup chart.

PHASE

12

# PERFORMANCE ENGINE

BUILD PROGRESS

WHEN DAY 30 TO 35 | OWNER LEADER | TIER SYSTEMS

MISSION Make the search several times faster with classic computer science tools, and prove every gain with measurements.

## YOU WILL BUILD

- Represent sets of machines (patched, breached, firewalled) as integer bitmasks so union, test and copy are single operations
- Zobrist hashing: give every (machine, status) pair a random 64 bit number and XOR them, so a position hash updates in O(1) per move
- TranspositionTable: dictionary keyed by hash storing depth, value, bound type (exact, lower, upper) and best move, with a size limit and a replacement policy
- Killer move and history heuristics added to move ordering
- NumPy arrays for the distance table and vectorised operations inside the evaluation function
- Profile with cProfile and keep a before and after table for every optimisation
- Benchmark nodes per second at depth 6 on 100 positions: plain, Alpha Beta, plus table, plus ordering

## CONCEPTS IN EASY WORDS

**Bitmask.** A set stored in one integer: bit i is 1 if machine i is in the set. Testing membership or copying the whole set takes one machine instruction.

**Zobrist hashing.** Give every (machine, status) pair a random number and XOR them together. XOR is its own inverse, so one move changes the hash with just two XORs.

**Transposition table.** Different move orders reach the same position. Cache the result and reuse it. It is memoisation for game search, and the replacement policy decides what to evict when full, like a CPU cache.

**Measure first.** Optimise only what the profiler shows is slow. Guessing wastes days.

## DEFINITION OF DONE

- [ ] Collisions measured over 1 million random positions and the count reported
- [ ] Chosen moves identical with and without the table
- [ ] Nodes per second and time per move improved, shown in a before and after table
- [ ] Profile report saved in results/

## TRAPS TO AVOID

- Reusing an entry that was searched to a shallower depth than needed
- Mixing up the bound type, which returns wrong values

## MILESTONE TO DEMONSTRATE

Transposition table, bitmask states and profiler tables with measured speedups.

PHASE

13

BUILD PROGRESS

# AI LAB: CALIBRATION AND MCTS

WHEN DAY 38 TO 45 | OWNER MEMBER 2 | TIER LAB

MISSION Go beyond the syllabus: make the sensors trustworthy with calibration, test a second inference method, add a different style of attacker and rank every agent fairly.

## YOU WILL BUILD

- Calibration: reliability diagram and Brier score for the Naive Bayes probabilities; fix with Platt style scaling or binning written in NumPy
- ROC and precision recall curves; choose the alert threshold by minimum expected cost where a miss costs 5 times a false alarm
- Feature selection by mutual information; explain each alert by the features that pushed its log probability up the most
- particle.py: 500 particles as an alternative to exact Bayes filtering; compare error and speed as map size grows
- mcts_attacker.py: Monte Carlo Tree Search with UCB1 selection and random rollouts, as a challenger to Minimax
- elo.py: round robin ladder where every agent plays every other; update Elo ratings and report each with a bootstrap interval
- Learning curves: accuracy against training size, to show bias versus variance

## CONCEPTS IN EASY WORDS

**Calibration.** A model is calibrated when events it calls 70 percent likely really happen 70 percent of the time. Naive Bayes is often overconfident, and the belief tracker needs honest numbers.

**Particle filter.** Represent belief by many sampled guesses of the intruder position. Move them, weight them by the alerts, resample. It scales to huge spaces where exact tables explode.

**MCTS and UCB1.** Play many random games from a position, keep win counts in a tree, and pick branches by average win plus an exploration bonus. It needs no evaluation function.

**Elo rating.** Every agent has a rating. The winner takes points from the loser, more if the win was surprising. Many games give a stable ranking.

## DEFINITION OF DONE

- [ ] Brier score improves after calibration (chart)
- [ ] Particle filter error close to the exact filter, with a speed comparison
- [ ] MCTS attacker appears in the Elo ladder with a rating interval
- [ ] Any alert in the UI can show its top 3 contributing features

## TRAPS TO AVOID

- Calibrating on the same data used to train the classifier
- Giving MCTS too few rollouts and then calling it weak

## MILESTONE TO DEMONSTRATE

Calibrated sensors, particle filter, MCTS challenger and an Elo ladder.

PHASE

14

BUILD PROGRESS

# SECURITY REALISM

WHEN DAY 43 TO 47 | OWNER BOTH | TIER BOOST

MISSION Ground the simulation in real security engineering so every number and label has a defensible meaning. Everything stays a simulation: no real exploits, scans or traffic.

## YOU WILL BUILD

- docs/threat_model.md: assets, trust boundaries, attacker goals and STRIDE categories for the fictional company network
- Hardness from a CVSS style score: attack complexity, privileges required and exposure combine into a score that maps to hardness 1 to 3
- Machine roles (web server, workstation, database, domain controller, backup) with different value and exposure
- Attacker stage labels from the MITRE ATT&CK model: Initial Access, Lateral Movement, Privilege Escalation, Exfiltration, shown in tutorial captions
- Chokepoint analysis: articulation points and a minimum cut (max flow) find the machines every route must cross; used by the generator and by CSP value ordering
- Detection cost model: cost of a missed attack versus a false alarm, used for the alert threshold and reported in results
- Generator uses roles and zones so maps look like real networks: a DMZ, an internal zone and a vault

## CONCEPTS IN EASY WORDS

**Threat modelling.** Before defending, list what you protect, who attacks and how. STRIDE is a checklist: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege.

**CVSS style scoring.** Vulnerabilities get a numeric severity from simple factors such as how easy they are to reach and to exploit. We borrow the idea to derive hardness.

**Kill chain and ATT&CK.** Attacks happen in stages. Naming the stage of each attacker move makes the game readable to any security engineer.

**Minimum cut.** The smallest set of links or machines whose removal separates entry from crown jewel. Guarding it is the most efficient defense.

## DEFINITION OF DONE

- [ ] Threat model written and reviewed by both members
- [ ] Hardness values trace back to a documented scoring rule
- [ ] Chokepoints match a hand worked example on a small map
- [ ] Detection cost shown in the results and tutorial

## TRAPS TO AVOID

- Treating the scores as real vulnerability data: they are a teaching model
- Adding real exploit details; keep everything abstract

## MILESTONE TO DEMONSTRATE

Threat model, CVSS style hardness, stage labels, chokepoint analysis and cost model.

PHASE 15

# INTEGRATION: MODES AND POLISH

BUILD PROGRESS

WHEN DAY 46 TO 52 | OWNER BOTH | TIER CORE

MISSION Turn the parts into one product with four game modes and a tutorial that explains the AI in plain words.

## YOU WILL BUILD

- Main menu: mode, map (random or evolved), difficulty (search depth, sensor accuracy)
- Modes: Human vs AI (either side), AI vs AI watch mode with speed slider, Human vs Human hot seat, Tutorial mode
- Tutorial captions generated by the AI itself, for example: the attacker ran A* and chose this route; the defender believes machine 7 is compromised with 63 percent
- Side panels from the reference: Agent Sensors PEAS, Genetic Algorithm, Naive Bayes Traffic Classifier, Defender Command
- Replay save and load, pause, step one turn
- Time budget guard: if an AI runs out of time, use the last finished depth
- Results screen that reads from the SQLite database, and a replay viewer that scrubs through any saved game

## CONCEPTS IN EASY WORDS

**One agent interface.** Every player, human or AI, implements choose_action(observation). The game never cares who is behind it, so any mix of players works.

**State machine for screens.** Menu, game, tutorial and results are states with clear transitions.

**Explainable AI.** A system that explains its choices is easier to trust, to debug and to explain.

**Observer pattern.** The UI subscribes to game events instead of polling the game. Add a new panel and the rules never change.

## DEFINITION OF DONE

- [ ] All four modes start from the menu
- [ ] 100 AI vs AI games run without a crash or freeze
- [ ] Tutorial mentions every technique at least once
- [ ] Three minute demo runs smoothly twice in a row

## TRAPS TO AVOID

- Menu code that reaches into the rules; keep screens thin
- Adding features instead of rehearsing the demo

## MILESTONE TO DEMONSTRATE

Menu with four modes, tutorial captions and all reference panels.

PHASE

16

BUILD PROGRESS

# EXPERIMENTS, REPORT, DEMO

WHEN DAY 52 TO 58 | OWNER BOTH | TIER FINAL

MISSION Prove everything with numbers, then package the project for submission and presentation.

## YOU WILL BUILD

- tournament_pool: headless runner for 9 matchups plus MCTS over 300 test maps and 3 sensor seeds, every game saved to the database
- charts.py reads from SQLite and draws the ten charts from the Experiments page in the document theme
- ablation.py: remove belief, CSP, pruning, transposition table, smoothing, calibration, tuned weights and evolved maps, one at a time
- Elo ladder with bootstrap intervals, and a paired significance test between the top two attackers
- Final README with screenshots, architecture, how to run and results
- Four minute demo video and a one page project brief

## CONCEPTS IN EASY WORDS

**Fair experiment.** Same maps, same seeds, same time limits for everyone; report the mean and an interval.

**Bootstrap interval.** Resample your results thousands of times to see how much a number could move. It gives an honest error bar without heavy theory.

**Ablation.** Switch one part off and measure the loss. It shows which ideas truly matter.

**Honest reporting.** If something is weak, say so and explain why. Readers trust that more than perfect looking numbers.

## DEFINITION OF DONE

- [ ] Every claim in the report points to a chart, table or SQL query
- [ ] Tests pass on a fresh clone
- [ ] Demo rehearsed and timed
- [ ] Repository tagged v2.0

## TRAPS TO AVOID

- Cherry picking seeds; always report every run
- Charts without axis labels and units

## MILESTONE TO DEMONSTRATE

Ten charts, Elo ladder, ablations, README, demo video and a one page project brief.

# 07 EXPERIMENTS AND PROOF

What we run, what we measure, what charts we ship

## TWELVE MATCHUPS, 300 TEST MAPS, 3 SENSOR SEEDS EACH: 900 GAMES PER CELL

| | Random Defender | Static Defender | Belief plus CSP |
|---|---|---|---|
| Random Attacker | baseline | baseline | baseline |
| Greedy A* Attacker | compare | compare | compare |
| Minimax Alpha Beta | compare | compare | main result |
| MCTS challenger | compare | compare | compare |

## TEN CHARTS TO SHIP

| # | CHART | STORY IT TELLS |
|---|---|---|
| 1 | Nodes expanded: Minimax versus Alpha Beta, depths 2 to 6, log scale | Pruning saves most of the search with identical moves |
| 2 | Heat map of attacker win rate over the matchup grid | A smarter defender lowers attacker win rate |
| 3 | CSP backtracks versus map size for four solver settings | MAC with MRV wins |
| 4 | Confusion matrix and 5 fold accuracy, with and without Laplace smoothing | Smoothing prevents zero probability failures |
| 5 | Probability on the true machine per turn, belief tracker versus uniform | Bayesian tracking finds the intruder |
| 6 | GA fitness per generation and difficulty of evolved versus random maps | Evolution produces fairer maps |
| 7 | Local search score per iteration, tuned versus default weights on test seeds | Tuning generalises |
| 8 | Tournament speedup on 1, 2 and 4 cores; nodes per second before and after each optimisation | Concurrency and caching pay off |
| 9 | Reliability diagram and Brier score before and after calibration | Probabilities become honest |
| 10 | Elo ladder with bootstrap intervals for every agent, MCTS included | A fair ranking of all agents |

## METRICS

- Win rate with a 95 percent bootstrap interval
- Average turns to breach
- Nodes expanded, nodes per second, milliseconds per move
- CSP assignments tried and backtracks
- Accuracy, precision, recall, F1, Brier score
- Probability on the true machine, top 3 hit rate
- Elo rating with interval, speedup across cores

## ABLATIONS: SWITCH ONE PART OFF

- Remove the belief tracker (uniform belief)
- Remove the CSP (static placement)
- Remove pruning and the transposition table
- Remove Laplace smoothing and calibration
- Default instead of tuned weights
- Random maps instead of evolved maps

# 08 ENGINEERING RULES AND DEMO

Habits that keep the build clean, and the four minute story

## ENGINEERING RULES

- One branch per phase, one pull request, reviewed by your partner before merge
- Tag every finished phase so you can always roll back to a working game
- Write the unit test the same day as the code
- Type hints and dataclasses on every public function; tests run on every push
- Never tune anything on the test seeds, only on development seeds
- Every slow function gets a profile line and a before and after number
- Keep the README current with one screenshot per phase

## RISKS AND FIXES

| RISK | FIX |
|---|---|
| **Minimax too slow in Python** | Move ordering, transposition table, depth 4 fallback, iterative deepening with a time budget |
| **Window freezes while the AI thinks** | Run the AI in a worker process and poll results each frame |
| **Windows starts workers by re importing** | Guard entry code with a main check; keep workers importable and free of global state |
| **Database is locked** | One writer process, WAL mode, batched transactions |
| **Classifier alerts too weak** | Tune threshold on validation data, calibrate, add an oracle sensor mode to separate the parts |
| **Scope creep** | Follow your route: A first, then B, then C |

## FOUR MINUTE DEMO SCRIPT

| TIME | WHAT YOU SHOW |
|---|---|
| 0:00 | One sentence pitch: two AIs fight over a hidden network intrusion |
| 0:20 | Human vs AI: show the intruder moving, the heatmap and alerts, place a honeypot |
| 1:00 | Toggle the A* path and the Alpha Beta pruning overlay, read the nodes expanded counter |
| 1:40 | Show the CSP panel and the solver comparison chart |
| 2:10 | Show the classifier, the calibration chart and the matchup heat map |
| 2:50 | Replay a saved game from the event log and run one SQL query on the results |
| 3:20 | Show the multi core speedup chart and the Elo ladder |
| 3:45 | Close: thirteen syllabus topics and a full engineering platform in one system, all measured |

## START NOW

Phase 00 takes one hour. Create the repository, the environment with pygame, numpy and matplotlib, the folder layout from the Architecture page, and a blank Pygame window. Then say "Phase 1" and we begin Phase 01 together: the rules engine, line by line.